

These options specify that Memcached is run as a daemon, with a max memory consumption of 1 GB. As the 'memcache' user, listen on the IP address 127.0.0.1 and port 11211. Alternatively, you might want to try running it with a `-vv` option in the foreground, to see what's happening behind the scenes:

```
memcached -vv -m 1024 -u memcache -l 127.0.0.1 -p 11211
```

Installing the Memcached client and using it largely depends on the language/environment you are using it on, so you can look up the documentation for that particular environment if you need help. As of now, we will just connect with a telnet session, and look at some statistics with the `stats` command. I'd rather not go into programming, and will keep this discussion more language-agnostic.

```
$ telnet 127.0.0.1 11211
Trying 127.0.0.1...
Connected to 127.0.0.1.
Escape character is '^['.
stats
```

After you enter the `stats` command as shown above, you will see a long list of statistics related to the cache, which include uptime, cache hits and cache misses, among others. This will resemble what is shown below:

```
STAT pid 1233
STAT uptime 23530
...
STAT get_hits 0
STAT get_misses 0
STAT delete_misses 0
...
END
```

The ten commandments for good cache performance

Here are some pointers that I'd like you to keep in mind while working with Memcached.

- 1) Take care of stale data and expiry times. Make sure that when a value is updated in the back-end database, the new value is also updated in the cache. Also, the data expiry time should be optimum; making it too long will unnecessarily take up more memory.
- 2) Always keep in mind that Memcached is not a database. Do not think that a value, once stored, will stay there until it is deleted.
- 3) Avoid storing frequently updated data in Memcached. There is no point in storing something that becomes stale frequently and has to be repeatedly retrieved from the database anyway.
- 4) Memcached clients in multiple languages might work

at the same time on the same server, but take care of compatibility and some features that a particular client might not support. For example, some of them might not support compression, or might perform compression in a different manner.

- 5) Use weights to configure multiple servers with varying amounts of memory available. Give more weight to servers with more memory, so that a larger share of the overall data is stored on them. Weighing can be assigned while populating the client with the servers available.
- 6) One important aspect of Memcached is that it has no in-built security mechanisms. It is your responsibility to set up a firewall, so that the Memcached interface is not publicly exposed.
- 7) It is recommended that you warm up the cache (load the frequently accessed data initially) before publicly deploying your Web application with a separate program or script, so that you hit the optimum performance levels immediately.
- 8) Each `get` or `set` operation fires a new atomic connection to the Memcached server, so it's much better if you have predefined a fixed number of connections in the connection pool.
- 9) While using Memcached in Linux, never allow it to use the swap, and never assign it more memory than the actual physical RAM available. That results in a very bad performance—there isn't any point of even having a cache, in such a situation.
- 10) Always try to cache complex processed data like objects or even HTML snippets, rather than raw data coming straight from the database.

To conclude, setting up a Memcached cluster is a very good way of improving performance if you have some spare resources available. In case you are working on a cloud, it is a must-have component since it can give you a significant performance boost and help you get rid of most of the bottlenecks in the application. 

Further reading

- [1] An introduction to Memcached—<http://www.search-this.com/2007/07/24/an-introduction-to-memcached/>
- [2] Memcached Internals—<http://www.adayinthelifeof.nl/2011/02/06/memcache-internals/>
- [3] Memcached protocol—<http://code.sixapart.com/svn/memcached/trunk/server/doc/protocol.txt>
- [4] Consistent hashing in Memcached—<http://www.mikeperham.com/2009/01/14/consistent-hashing-in-memcache-client/>

By: Ankit Mathur

The author has a crush on Java, and also loves flirting with almost everything related to databases, cloud computing and Web technologies. Feel free to poke fun at this article, and direct your feedback to ankitloaded@gmail.com.